

SystemVerilog Section 6

Randomization

This section continues the data types and OOP revision guide. It explains the randomization practice supplied on **26 September 2026**, including the final distribution, implication, equivalence, conditional-address, and constraint-mode example. The saved course examples in Parts 22-29 provide supporting context.

Each main concept starts with a **Definition to say aloud**, followed by the mechanism, a worked example, and the correction needed in your practice. Read in order to build the model; use the contents and PDF bookmarks for revision. The pronunciation guide in **6.12** shows how to read the keywords, operators, and complete statements aloud. Section numbers such as **6.7** belong to this guide; **Part 29** is a separate saved course-example number.

Your [original practice](#) is preserved unchanged. It contains several independent programs that reuse generator and `tb`; compile them separately. The complete examples below use distinct names and explicit failure checks. Shorter blocks are explanatory excerpts. **Derived** results are reasoning exercises; recorded simulator checks are identified in [6.13](#).

Contents

6.1 Transactions, random fields, and the solver	2
6.2 Randomization success, failure, and safe checking	3
6.3 Ranges, membership, and exclusions	5
6.4 randc, object lifetime, and cycle boundaries	7
6.5 External constraints and class scope	9
6.6 Callbacks and ranges that change at runtime	10
6.7 Distribution weights and probability	13
6.8 Implication and bidirectional solving	16
6.9 Equivalence and conditional addresses	17
6.10 Constraint modes and your final program	20
6.11 A method for debugging randomization	23
6.12 Pronunciation, oral revision, and worked answers	25
6.13 Source map and verification	29

6.1 Transactions, random fields, and the solver

6.1.1 What a transaction represents

Definition to say aloud. A transaction is an object that groups the data and control information for one testbench operation, together with any associated expected or observed results.

Your opening comment says that a transaction holds inputs and outputs. That is a useful starting point. In an adder example, inputs might be *a* and *b*, while the observed output is *y*. In a memory example, the same idea expands to read/write controls, an address, write data, and read data. A transaction groups related information so that each testbench component knows which operation it is handling.

The class name *generator* in your practice does not give the object a special language role. That class currently holds randomizable data, constraints, and display methods, so it behaves mainly as a transaction-like stimulus object. In a larger testbench, a separate generator component usually creates or randomizes transactions and sends them to a driver.

```

generator -> transaction -> driver -> DUT
                        |
                        v
                        monitor -> scoreboard
  
```

The driver converts transaction fields into interface activity. The monitor reconstructs what actually happened at the interface. The scoreboard compares observed behavior with a prediction. This explains your `gen -> drv -> dut -> monitor -> scoreboard` comment, while keeping the roles distinct: a call to `randomize()` alone does not drive pins, wait for a clock, sample an output, or verify a DUT.

6.1.2 What `rand` changes in a declaration

Definition to say aloud. `rand` marks a class property as eligible for assignment by the object's constraint solver when that property is active during randomization.

Pronunciation. Say `rand` as "rand," rhyming with "hand." Say "randomize" as **RAN-duh-mize** and "constraint" as **kuhn-STRAYNT**.

Your declaration is:

```

rand bit [3:0] a, b;
bit [3:0] y;
  
```

Both *a* and *b* are unsigned, four-bit, two-state variables. Each can represent sixteen values, from 0 through 15. `rand` does not allocate the object and does not continuously change the variables. Before the first successful randomization, these two-state fields have their normal default value of zero unless initialized or assigned otherwise.

y has the same bit representation but lacks `rand`, so an ordinary `g.randomize()` does not choose a new value for it. In your pasted programs, nothing assigns a DUT result to *y*; it therefore stays at its initialized/default value. Do not interpret it as a verified output merely because it is stored in the same object.

For an unsigned four-bit adder, the largest mathematical sum is $15+15=30$. A four-bit `y` cannot retain that full result. A five-bit output or expected result is needed when carry matters. Widen the operands explicitly when illustrating that calculation:

```
bit [4:0] expected;
expected = {1'b0, a} + {1'b0, b};
```

This is a reference-model calculation, not an assignment made automatically by the randomizer.

6.1.3 Constraints describe a set of legal solutions

Definition to say aloud. A constraint is a declarative relation that every selected randomized solution must satisfy while that constraint is active.

```
constraint data_a { a > 3; a < 7; }
constraint data_b { b == 3; }
```

For this example, `a` can be 4, 5, or 6, and `b` must be 3. Both expressions in `data_a` apply together. The solver does not execute `a > 3` first and later overwrite it with `a < 7`. It chooses values for all active random fields that satisfy the active relations simultaneously.

The same conjunction applies across named constraint blocks. Placing contradictory rules in separate blocks does not make one override the other. `constraint c1 { a == 4; }` combined with `constraint c2 { a == 5; }` gives no legal solution while both blocks are active.

Nonrandom members such as your `int min` and `int max` supply fixed state for that particular solve. The solver can change active `a` and `b`; it cannot silently adjust ordinary `min` and `max` to rescue an impossible range. A procedural setter changes those bounds before the next solve.

Reference: [IEEE 1800-2017](#), §§6.8, 6.11, 18.3-18.6.

6.2 Randomization success, failure, and safe checking

6.2.1 What randomize returns

Definition to say aloud. `randomize()` returns 1 when it assigns a legal solution to the selected active random fields, and returns 0 when the randomization fails.

The return value is a status, not the generated value. Your commented `status = g.randomize();` is a valid way to separate the status from the stimulus. Allocate the object first, then check the result before using the new transaction:

```
generator g;
initial begin
    bit success;
    g = new();
    success = g.randomize();
    if (!success)
        $fatal(1, "Randomization failed at time %0t", $time);
    // Consume g.a and g.b only after success.
end
```

An unconstructed handle is `null`. Calling `g.randomize()` through a null handle is an object-access error, not an ordinary unsatisfiable-constraint result. Likewise, malformed syntax or duplicate class definitions fail before a

usable randomization call exists. Distinguish compilation, object construction, and solving when diagnosing a failure.

6.2.2 Why a four-bit variable can make a constraint impossible

Your commented constraint `a > 16` has no solution for bit `[3:0]` `a`: even 16 itself is outside the type's representable set, and the constraint asks for something larger. The solver cannot increase `a`'s width.

These are three different failure mechanisms:

Situation	Reason no solution exists
Four-bit <code>a</code> , with <code>a > 16</code>	The type supplies only 0 through 15.
<code>a inside {[0:3]}</code> and <code>a inside {[12:15]}</code>	The two allowed sets have an empty intersection.
<code>min=12, max=8</code> , with <code>a inside {[min:max]}</code>	A reversed numerical membership interval is empty.

The range notation in `inside` denotes numerical lower and upper bounds. It is different from the descending bit-index declaration `[3:0]`. Writing `[12:8]` inside a membership set does not reverse an enumeration and generate 12, 11, 10, 9, 8.

The language rule for a failed solve is that the random variables retain their previous values. `post_randomize()` is not called. Code executed in `pre_randomize()` can already have changed other state, so failure is not a general rollback of every effect inside the object. The distinction matters when counters or setup functions run before the solver. The local XSim check exposed a discrepancy with field preservation; [6.13.3](#) records it separately from this language rule.

6.2.3 Why the failure handler must stop or skip consumption

Your practice sometimes uses:

```
assert(g.randomize()) else $display("Randomization failed");
g.post_randomized();
```

The display reports a problem but does not stop the next statement. Your manually called `post_randomized()` can still print stale values after a failure. A loop may then appear to have generated valid new stimulus even though it did not.

For stimulus generation, use an explicit condition:

```
if (!g.randomize())
    $fatal(1, "Cannot generate legal stimulus");
```

The `assert(g.randomize())` idiom is common in small lessons, but putting the operation itself inside an assertion makes stimulus generation dependent on assertion evaluation being enabled. An explicit `if` makes the required call unconditional with respect to assertion controls. `$fatal` terminates with an error indication; `$finish` ends simulation normally unless the surrounding workflow interprets it otherwise. Reporting an error and continuing can be appropriate in a deliberately designed negative test, but that test must avoid consuming failed stimulus.

6.2.4 The declaration-order question in your comment

Your opening example places `int status = 0;` after `g = new();` in the same `begin/end` block. Declarations in that block must precede its executable statements. Move the declaration to the beginning, or introduce a nested block with its own declarations first.

```
initial begin
  int status;
  g = new();
  status = g.randomize();
  if (status == 0)
    $fatal(1, "No legal stimulus");
end
```

This fixes a compile-time ordering error; it does not change the constraint solution space. A `for (int i=0; ...)` declaration is part of the `for` construct and has its own permitted syntax.

Reference: [IEEE 1800-2017](#), §§9.3.1, 18.6.1-18.6.3.

6.3 Ranges, membership, and exclusions

6.3.1 The exact meaning of inside

Definition to say aloud. `inside` tests whether a value belongs to any member or inclusive range in a set; in a constraint, that membership must hold for the chosen value.

Your commented alternatives contain:

```
constraint data {
  a inside {[0:8], [10:11], 15};
  b inside {[3:11]};
}
```

The comma-separated items for `a` form a union. Values 0 through 8 are included, then 10 and 11, then 15. The resulting set has twelve values. Values 9, 12, 13, and 14 are excluded. For `b`, both endpoints are included, giving nine values, 3 through 11.

Within one set, commas mean alternatives. Between the two semicolon-terminated constraint expressions, both requirements apply. Because these two fields have no relation tying them together, this example has $12 \times 9 = 108$ legal pairs. Counting pairs is useful because a constraint solver chooses a combination of values, and a bias in combinations can affect individual-variable frequencies.

6.3.2 What your active exclusions permit

Your first active class uses the complement of two intervals:

```
constraint data {
  !(a inside {[3:7]});
  !(b inside {[5:9]});
}
```

Start with the type's full domain before taking the complement. Here it is 0 through 15, so the legal values are:

Field	Removed values	Remaining values
a	3, 4, 5, 6, 7	0, 1, 2, 8, 9, 10, 11, 12, 13, 14, 15
b	5, 6, 7, 8, 9	0, 1, 2, 3, 4, 10, 11, 12, 13, 14, 15

Each field has eleven legal values, so there are $11 \times 11 = 121$ legal pairs. Negative values are not newly permitted by the negation: the unsigned four-bit type never allowed them. The parentheses make it clear that ! negates the complete membership test.

These fields are declared rand in this first program. Repeated values are legal. Ten samples cannot visit all eleven possible values of either field, and the language does not promise that those ten samples are different.

6.3.3 Complete example: checking exclusions and simulation time

This version retains the legal sets from your first program. It adds explicit time units and failure checks. The object is created once because independent object snapshots are not needed for this isolated printing example.

```
`timescale 1ns/1ps
class exclusion_item;
  rand bit [3:0] a, b;
  bit [3:0] y;
  constraint legal {
    !(a inside {[3:7]});
    !(b inside {[5:9]});
  }
endclass

module exclusion_demo;
  timeunit 1ns; timeprecision 1ps;
  initial begin
    exclusion_item g;
    g = new();
    $timeformat(-9, 0, " ns", 8);
    for (int i=0; i<10; i++) begin
      if (!g.randomize()) $fatal(1, "Excluded-range solve failed");
      if ((g.a inside {[3:7]}) || (g.b inside {[5:9]}))
        $fatal(1, "Excluded value was generated");
      if (g.y != 0) $fatal(1, "Nonrandom y changed");
      $display("i=%0d a=%0d b=%0d t=%0t", i, g.a, g.b, $time);
      #10;
    end
    $display("PASS exclusion_demo");
    $finish;
  end
endmodule
```

Derived timing. The first display is at 0 ns, followed by displays at 10, 20, and so on through 90 ns. The last #10 advances the final PASS to 100 ns. randomize() is a function and does not advance simulation time; the explicit delay does that. Your original fragment has no local time-unit declaration, so its #10 depends on the enclosing compilation/simulator time settings.

The random values are intentionally not predicted. A correct result is any sequence satisfying the exclusions, with y unchanged and the stated timing. Hard-coding an attractive-looking random sequence would test a particular generator run rather than the language property.

Reference: [IEEE 1800-2017](#), §§11.4.13, 18.5.3, 18.6.3.

6.4 randc, object lifetime, and cycle boundaries

6.4.1 What cyclic randomization guarantees

Definition to say aloud. `randc` makes a random variable traverse a randomized cycle of its legal values without repeating within that cycle; after the cycle finishes, another cycle begins.

Read aloud. `randc bit [3:0] a` is "rand-see, bit three down to zero, a." When explaining the meaning, say "a four-bit unsigned random-cyclic variable." "Cyclic" is **SY-klik**.

For a single independent `randc bit [1:0] a` under unchanged constraints, a possible illustrative sequence is:

```
Cycle 1:  2  0  1  3
Cycle 2:  3  1  0  2
           ^ each cycle is a separate permutation
```

The two adjacent 3s at the boundary are legal. The rule is no repeat *within a cycle*, not no adjacent repeat anywhere in the simulation. Likewise, an arbitrary sliding window crossing two cycles need not contain every value once.

`rand` has no corresponding cycle guarantee. A legal `rand` value can repeat immediately or remain unseen for many draws. With `randc`, the legal set and the persistent cycle state both matter. IEEE 1800-2017 §18.4.2 specifies recomputing the permutation when the constraints change or no remaining permutation value satisfies them. A literal implementation bucket is only a teaching analogy; it is not an object you access from your class.

6.4.2 Why new inside the loop defeats the intended experiment

Your third and fourth programs repeatedly allocate the object:

```
for (int i=0; i<10; i++) begin
  g = new();
  // set bounds, then randomize once
end
```

For the ordinary nonstatic members in your classes, each new object begins with its own randomization state and cycle history. You are taking one sample from ten objects. You are not taking ten successive samples from one continuing cyclic sequence. Values may repeat between objects without violating `randc`.

Move construction outside the loop when the goal is to observe a cycle:

```
g = new();
for (int i=0; i<10; i++) begin
  // randomize the same object repeatedly
end
```

Keeping one generator object does introduce an OOP consideration in a larger testbench. If you send the same handle to another component and then randomize that object again, the recipient can observe the updated object instead of the original sample. Preserve persistent generation state, and send a fresh transaction or suitable copy for each operation. A mailbox of handles does not clone objects automatically.

Recreating an object is therefore appropriate when distinct transactions are needed, but it changes what a `randc` experiment measures. Static random members have shared state and are a separate case; your pasted declarations are not static.

6.4.3 Two randc fields do not cover every pair

In `randc bit [3:0] a, b;`, `a` and `b` each have their own cyclic behavior. The pair is not one eight-bit cyclic variable. If each field is independently restricted to ten values, a ten-call cycle gives ten observations of pairs, while there are one hundred possible pairs. It cannot cover the full cross product in those ten calls.

For example, `a` can visit 2 through 11 once and `b` can also visit those values once, yet many `(a,b)` combinations never appear. The two fields may even be equal on a particular call: separate `randc` declarations do not assert `a != b`.

The language solves `randc` variables before ordinary `rand` variables. Relationships between several cyclic fields can introduce additional solving restrictions, so do not extrapolate the independent-field example to every correlated model. The tests here deliberately use independent cyclic fields with simple fixed ranges.

6.4.4 Your SPACE 1 and SPACE 2 loops

Both phases in your fifth program call `pre_randomized(2,11)`, and both repeat ten times on the same object. The legal set is unchanged. Reassigning the same bounds is not a change in the effective range and does not, by itself, restart the cycle.

There are ten legal values. With this independent setup, the first ten successful calls complete one cycle for each field, and the next ten complete another. Seeing a value in SPACE 2 that appeared in SPACE 1 is expected. Those displays do not demonstrate a changed range because the arguments are identical.

6.4.5 Complete example: verify complete cycles

The masks below record values seen during each ten-sample cycle. Bit zero represents value 2; bit nine represents value 11. The test checks properties of the samples and does not require any specific permutation.

```
class cyclic_item;
  randc bit [3:0] a, b;
  constraint legal {
    a inside {[2:11]};
    b inside {[2:11]};
  }
endclass

module cyclic_demo;
  initial begin
    cyclic_item g;
    bit [9:0] seen_a, seen_b;
    g = new();
    for (int cycle=0; cycle<2; cycle++) begin
      seen_a = '0;
      seen_b = '0;
      repeat (10) begin
        if (!g.randomize()) $fatal(1, "Cycle solve failed");
        if (!(g.a inside {[2:11]}) ||
            !(g.b inside {[2:11]}))
          $fatal(1, "Value outside cycle range");
        if (seen_a[g.a-2] || seen_b[g.b-2])
          $fatal(1, "Repeated value within a cycle");
        seen_a[g.a-2] = 1;
        seen_b[g.b-2] = 1;
      end
    end
  end
endmodule
```

Code continued

```

    if (seen_a != 10'h3ff || seen_b != 10'h3ff)
        $fatal(1, "Incomplete cycle");
    end
    $display("PASS cyclic_demo: two complete cycles per field");
    $finish;
end
endmodule

```

Recall check. Moving `g = new()` into the repeat loop invalidates the premise of this check. A duplicate might then occur, but a particular short run could also happen to avoid duplicates. A demonstration of repeated values is evidence about that run; the absence of a repeat in a small sample is not proof of shared cycle history.

Reference: [IEEE 1800-2017](#), §§8.12, 18.4.2; [Accellera SV-EC discussion](#) on cyclic-variable ordering.

6.5 External constraints and class scope

6.5.1 What extern declares

Definition to say aloud. An external constraint separates a class constraint's declaration from its body; the qualified definition still belongs to that class.

Your class declares `extern constraint data;`, and the later block defines `constraint generator::data { ... }`. The `::` operator is class-scope qualification. It does not construct an object and does not turn the constraint into global procedural code.

The declaration and definition must match. An explicitly `extern` constraint needs its definition, in the same enclosing scope as the class and after the class declaration. Define it once. End the constraint body with `}`. A following semicolon is unnecessary for the constraint syntax; the cleaned examples omit it instead of relying on acceptance as a separate empty construct.

Your external `display()` function follows a related pattern: an `extern function void display();` prototype in the class and `function void generator::display();` outside. It remains a method and can directly access `a` and `b` in the object on which it is called.

6.5.2 Two variants in your pasted work

The second program uses ordinary `rand` fields with `a` in 0 through 3 and `b` in 12 through 15. Those are two independent four-value sets, making sixteen legal pairs. The fourth program uses `randc`, expands `a` to 0 through 4, and keeps `b` in 12 through 15. The legal sets then have five and four values respectively.

If the fourth program reused one object, `a` and `b` would complete their independent cycles after different numbers of calls. In the supplied version, `new()` is inside the loop, so it does not observe either continuing cycle. Moving the body outside the class has no role in that behavior; the changed modifier and object lifetime do.

6.5.3 Complete example: an external body and method

```

class external_item;
  rand bit [3:0] a, b;
  extern constraint legal;
  extern function void display();
endclass

constraint external_item::legal {
  a inside {[0:3]};
  b inside {[12:15]};
}

function void external_item::display();
  $display("external: a=%0d b=%0d", a, b);
endfunction

module external_demo;
  initial begin
    external_item g;
    g = new();
    repeat (10) begin
      if (!g.randomize()) $fatal(1, "External solve failed");
      if (!(g.a inside {[0:3]}) ||
          !(g.b inside {[12:15]}))
        $fatal(1, "External constraint was not satisfied");
      g.display();
    end
    $display("PASS external_demo");
    $finish;
  end
endmodule

```

In your original second program, a display is followed by #10, then g.display(). No intervening operation randomizes or assigns a and b, so the second display reports the same values ten time units later. The external method does not trigger a new solve.

Reference: [IEEE 1800-2017](#), §§8.24, 18.5.1.

6.6 Callbacks and ranges that change at runtime

6.6.1 The exact callback names matter

Definition to say aloud. pre_randomize() is an automatic function hook before solving; post_randomize() is an automatic function hook after a successful solve and assignment.

Your practice defines pre_randomized(input int min, input int max) and post_randomized(). The extra final **d** makes these ordinary user methods. They run because your testbench calls them explicitly. They are not misspelled callbacks that a simulator somehow recognizes by intent.

Method form	How it runs
<code>function void pre_randomize();</code>	Automatically before a <code>randomize()</code> attempt.
<code>function void post_randomize();</code>	Automatically after successful randomization.
<code>pre_randomized(min,max)</code>	Only when explicitly called; your range setter.
<code>post_randomized()</code>	Only when explicitly called; your display helper.

The automatic callbacks have no arguments and return `void`. A clear naming scheme is `set_range(low,high)` for explicit setup, `display()` for an explicitly requested print, and the exact callback names only when automatic invocation is intended.

If you rename `post_randomized()` to `post_randomize()` but leave the explicit call in the testbench, the function can run twice after a successful solve: once automatically and once manually. If it computes a checksum, this may merely duplicate work; if it increments a transaction count, the count becomes wrong.

6.6.2 Why this.min is needed

In function `void pre_randomized(input int min, input int max);`, the argument names hide the class member names within the function. `this.min = min;` writes the current object's member using the supplied argument. Writing only `min = min;` assigns the argument to itself and leaves the object unchanged.

Renaming the parameters also removes the ambiguity:

```
function void set_range(int low, int high);
    min = low;
    max = high;
endfunction
```

`min` and `max` are ordinary signed 32-bit `int` members. Their initial value is zero when you do not initialize or assign them. The phrase in your comment, "min / max is a 2 state logic," is better stated as: `int` **is a two-state integral type and these members default to zero**. It is not the four-state type named `logic`.

6.6.3 Your call `set_range(12,33)` does not create values above 15

The legal domain is the intersection of what the variable can represent and what the constraint allows. With `randc bit [3:0] a` and `a inside {[min:max]}`, the relevant nonnegative cases are:

Bounds at the call	Effective legal values for four-bit a
No setup; defaults 0 and 0	Only 0.
12 and 33, as in your code	12, 13, 14, 15.
2 and 11	2 through 11: ten values.
16 and 33	None; no representable value fits.
12 and 8	None; the interval is empty.

An upper bound of 33 does not get assigned to `a`; it is a bound used in a comparison. It neither widens the field nor means "generate 33 and truncate it to four bits." Signedness and mixed-width expressions deserve additional care for negative bounds; all the practice intervals above are nonnegative and avoid that ambiguity.

Because 12 through 15 is a nonempty set, your (12,33) call is a valid range constraint for this field. If you retain the object across calls, that independent `randc` field cycles through four legal values. If you recreate the object every time, the cross-call cycle guarantee is lost.

6.6.4 Changing the range versus repeating the same assignment

Setting (2,11) again does not change the set. Setting (12,15) after (2,11) does change it. On the next call the active constraint reads the new state, so the old interval no longer governs that call.

When legal sets overlap, a value seen before a constraint change may appear again after it. Do not use cyclic history as a global "never generated before" database across changing domains. If a test needs global uniqueness, it needs an explicit history policy; that is an additional requirement beyond `randc`.

6.6.5 Complete example: callbacks, failure, and recovery

This example keeps your two cyclic fields and dynamic bounds, replaces the helper names with clear roles, and uses automatic callbacks to count attempts and successes. `y` is widened to hold the computed five-bit sum. It is a derived expected value here, not a sampled DUT output.

```
class range_item;
    randc bit [3:0] a, b;
    bit [4:0] y;
    int min, max;
    int attempts, successes;
    function void set_range(int low, int high);
        min = low;
        max = high;
    endfunction
    constraint legal {
        a inside {[min:max]};
        b inside {[min:max]};
    }
    function void pre_randomize();
        attempts++;
    endfunction
    function void post_randomize();
        successes++;
        y = {1'b0, a} + {1'b0, b};
    endfunction
endclass

module range_demo;
    initial begin
        range_item g;
        bit [3:0] old_a, old_b;
        bit [4:0] old_y;
        g = new();
        if (!g.randomize()) $fatal(1, "Default range failed");
        if (g.a != 0 || g.b != 0 || g.y != 0)
            $fatal(1, "Default bounds should allow only zero");

        g.set_range(12, 33);
        if (!g.randomize()) $fatal(1, "12 to 33 should be legal");
        if (!(g.a inside {[12:15]})) ||
            !(g.b inside {[12:15]}))
            $fatal(1, "Four-bit domain was not respected");
        if (g.y != ({1'b0,g.a} + {1'b0,g.b}))
            $fatal(1, "Derived sum is wrong");
        old_a = g.a; old_b = g.b; old_y = g.y;
    end
endmodule
```

Code continued

```

    g.set_range(16, 33);
    if (g.randomize()) $fatal(1, "Impossible range succeeded");
    if (g.a != old_a || g.b != old_b || g.y != old_y)
        $fatal(1, "Failed solve unexpectedly changed the data");
    if (g.attempts != 3 || g.successes != 2)
        $fatal(1, "Callback success/failure counts are wrong");

    g.set_range(2, 11);
    if (!g.randomize()) $fatal(1, "Recovery failed");
    if (!(g.a inside {[2:11]}) ||
        !(g.b inside {[2:11]}))
        $fatal(1, "Recovery used the wrong interval");
    if (g.attempts != 4 || g.successes != 3)
        $fatal(1, "Recovery callback counts are wrong");
    $display("PASS range_demo: attempts=4 successes=3");
    $finish;
end
endmodule

```

Derived call sequence. Attempt 1 uses the default interval and succeeds. Attempt 2 uses the representable portion of 12 through 33 and succeeds. Attempt 3 fails, yet its `pre_randomize()` has already incremented `attempts`; `post_randomize()` does not run. Attempt 4 uses the restored legal interval and succeeds. The final counts are therefore four attempts and three successes.

The deliberate failed solve may produce a simulator warning. The enclosing test treats that failure as the expected outcome and checks the unchanged fields. This is different from ignoring a failure in a stimulus loop.

Observed simulator limitation. XSim 2024.1 returned zero for the impossible interval and correctly skipped the post-hook, but changed `a` and `b`. This reference test therefore stops at its unchanged-data check on that simulator, before the recovery attempt. A separate diagnostic reproduced the issue with both `randc` and `rand`. The expected retention and recovery sequence above is the language-based prediction, not a passing XSim transcript. Stop or discard failed stimulus rather than relying on its values.

Keep constrained random fields unchanged in `post_randomize()` unless you deliberately intend to invalidate the solver's result. Assigning `a = 0` in that callback could violate an active `a inside {[12:15]}` rule after the solver has already finished. Use the callback for derived nonrandom data, bookkeeping, or observation, and keep it free of timing controls.

Reference: [IEEE 1800-2017](#), §§8.11, 18.4.2, 18.6.2-18.6.3, 18.10.

6.7 Distribution weights and probability

6.7.1 What dist adds to a membership constraint

Definition to say aloud. `dist` restricts a random expression to a listed set and assigns relative weights that influence how frequently its legal values are selected.

An isolated unconstrained `rand bit` gives equal probability to 0 and 1. `wr dist {0 := 30, 1 := 70};` expresses a 30:70 preference, which normalizes to 30% and 70% when no other constraint changes that distribution. The numbers are weights; they do not have to total 100. Weights 3 and 7 describe the same ratio.

An omitted value is excluded by this distribution expression. A listed value with zero weight is also excluded. Zero therefore has a stronger meaning than "unlikely."

The distribution must still satisfy all active hard relations. If other constraints make the requested weights incompatible, the language requires legal solutions and does not promise that all conflicting marginal weights will be preserved. Zero-weight exclusions remain constraints. This is why the complete `rst/ce` example needs more care than an isolated weighted bit.

6.7.2 The difference between `:=` and `:/`

Definition to say aloud. `:=` gives the stated weight to each value in a range; `:/` gives the stated weight to the range as a whole and divides it among its values.

Read aloud. Say `dist` as "dist," the first syllable of "distance." Read `:=` as "colon equals" and `:/` as "colon slash," then explain whether the weight belongs to each value or to the whole range.

For a singleton such as 0, the two operators have the same weighting effect. Your `wr dist {0 := 30, 1 := 70}` and `rd dist {0 :/ 30, 1 :/ 70}` therefore specify the same marginal preference when considered independently. That does not make `wr` and `rd` equal on every call; there is no equality relation between those variables.

Your two-bit fields make the range distinction visible:

```
rand bit [1:0] var1, var2;
constraint data {
  var1 dist {0 := 30, [1:3] := 70};
  var2 dist {0 :/ 30, [1:3] :/ 90};
}
```

For `var1`, the three range values each receive weight 70. The total weight is $30+3 \times 70=240$. Zero has probability $30/240=1/8=12.5\%$. Each of 1, 2, and 3 has probability $70/240=7/24$, about 29.17%. The entire nonzero group has probability $210/240=87.5\%$.

For `var2`, the range's total weight 90 is shared among three values, giving weight $90/3=30$ to each. All four values now have weight 30. Each has probability $30/120=1/4=25\%$. This line is uniform over the four values; it does not make each nonzero value 90% likely.

Value	var1 weight	var1 probability	var2 weight	var2 probability
0	30	12.5%	30	25%
1	70	about 29.17%	30	25%
2	70	about 29.17%	30	25%
3	70	about 29.17%	30	25%

These calculated probabilities apply to your independent first class with no additional restrictions on those fields. The commented note `[1:3] :/ 60` would give each of 1, 2, and 3 weight 20. To obtain actual probabilities, also include the weights of any other permitted values in the denominator.

6.7.3 Why fifteen calls do not prove a percentage

Your loop has fifteen iterations. A 70% target would correspond to $15 \times 0.7=10.5$ occurrences, which cannot be an exact integer count. The weight specifies a probability model; each finite run produces counts that fluctuate.

For independent draws of a Bernoulli event with probability p , the expected count is $N \times p$. A larger number of trials makes the measured fraction more informative, but it still does not turn a probabilistic preference into an

exact quota. If a requirement says "exactly seven writes in ten operations," design a sequence with that count; a 70:30 dist alone does not implement it.

Also separate distribution from coverage. A heavily weighted common operation can be generated many times while a rare corner remains unseen. Counting values tells you what this run exercised. It does not prove the DUT responded correctly, and it does not replace a scoreboard.

6.7.4 Complete example: observe a histogram

This version restores your commented first example as a separate runnable program and increases the sample count. It reports counts without asserting an exact histogram. The two control fields remain separate random variables.

```
class weighted_item;
  rand bit wr, rd;
  rand bit [1:0] var1, var2;
  constraint data {
    var1 dist {0 := 30, [1:3] := 70};
    var2 dist {0 := 30, [1:3] := 90};
  }
  constraint control {
    wr dist {0 := 30, 1 := 70};
    rd dist {0 := 30, 1 := 70};
  }
endclass

module distribution_demo;
  initial begin
    weighted_item f;
    int counts1[4], counts2[4];
    int writes, reads, total1, total2;
    f = new();
    repeat (12000) begin
      if (!f.randomize()) $fatal(1, "Distribution solve failed");
      counts1[f.var1]++;
      counts2[f.var2]++;
      writes += int'(f.wr);
      reads += int'(f.rd);
    end
    foreach (counts1[i]) begin
      total1 += counts1[i];
      total2 += counts2[i];
      $display("value=%0d var1_count=%0d var2_count=%0d",
        i, counts1[i], counts2[i]);
    end
    if (total1 != 12000 || total2 != 12000)
      $fatal(1, "Histogram accounting failed");
    $display("writes=%0d reads=%0d samples=12000", writes, reads);
    $display("PASS distribution_demo: sample accounting");
    $finish;
  end
endmodule
```

Derived expectations. At 12,000 samples, the expected counts for var1 are 1,500 for zero and 3,500 for each other value. The expected var2 counts are 3,000 each. The expected write and read counts are 8,400 each. Recorded counts will differ. The PASS marker verifies successful calls and count accounting; it is not a statistical certification of a random-number generator.

The standard prohibits applying `dist` to `randc` variables. Your weighted fields are correctly declared `rand`; the separate `randc a` in the final generator does not have a `dist` expression applied to it.

Reference: [IEEE 1800-2017](#), §18.5.4. The probabilities above are derived from your specific weights.

6.8 Implication and bidirectional solving

6.8.1 Read the relation as a truth rule

Definition to say aloud. An implication $A \rightarrow B$ requires B whenever A is true; when A is false, that implication adds no requirement on B .

Your relation is:

```
constraint control_rst_ce {
    (rst == 0) -> (ce == 1);
}
```

For the one-bit two-state fields in your class, its truth table is:

<code>rst</code>	<code>ce</code>	Antecedent <code>rst==0</code>	Is this pair legal?
0	0	true	No: the required <code>ce==1</code> is false.
0	1	true	Yes.
1	0	false	Yes.
1	1	false	Yes.

The Boolean equivalent is $(rst \neq 0) \mid\mid (ce == 1)$. The rule does not say that `ce` is zero when `rst` is one. Adding that behavior would require another relation or an `else` branch.

The signal name alone does not establish whether reset is active high or active low. Your code specifically constrains the case `rst == 0`. Explain that literal condition first, then attach protocol meaning only when the DUT's reset polarity is defined.

6.8.2 Logical implication still participates in joint solving

The arrow looks directional, but this is not procedural execution. The solver considers both random values together. If another constraint forces `ce == 0`, then the implication forces `rst == 1`, because $(0, 0)$ is forbidden. This is bidirectional reasoning about a one-way logical relation; it does not turn implication into equivalence.

Similarly, writing the constraint in an earlier or later block does not create an execution order. The order in which constraints appear in source is not a request to "pick reset first, then pick enable." `solve ... before ...` is a separate distribution-ordering construct.

The same `->` token can occur in procedural event-trigger syntax, but this block is a constraint context. It is also different from the temporal assertion operators `|->` and `|=>`. Your example relates values in one randomization solution and specifies no cycle delay.

6.8.3 Why the reset and enable weights are not a ten-call quota

Your final class combines:

```
rst dist {0 := 40, 1 := 60};
ce dist {1 := 80, 0 := 20};
(rst == 0) -> (ce == 1);
```

The hard implication always excludes $(rst, ce) = (0, 0)$. The two distributions express preferences among remaining solutions. Neither ten iterations nor the text of the weights proves that reset will be zero four times or enable will be one eight times. The weighted fields are connected by a constraint, so treating them as independent coin flips and multiplying their probabilities is not a portable derivation of the joint distribution.

In fact, the requested marginals are mathematically compatible with the implication: masses of 0.4 on $(0, 1)$, 0.2 on $(1, 0)$, and 0.4 on $(1, 1)$ would meet them. That is a demonstration of compatibility, not a recorded simulator result or a claim that the source mandates this particular implementation of joint sampling. Check legality deterministically and measure frequencies separately.

For a clean derivation of solution-space bias, temporarily omit both `dist` rules and consider only two one-bit `rand` fields with the implication. There are three equally weighted legal pairs in the unordered model. Only one has `rst==0`, so its probability is 1/3. If you add `solve rst before ce;`, the legal pairs stay the same but reset can be chosen with equal probability first, then enable is chosen from the legal values for that reset.

This small extension explains why relational constraints can change a field's marginal distribution. It does not belong in a `solve ... before ...` relation involving your `randc` a: `randc` variables are excluded from that ordering syntax.

Reference: [IEEE 1800-2017](#), §§11.4.7, 18.5.4, 18.5.6, 18.5.10.

6.9 Equivalence and conditional addresses

6.9.1 Equivalence constrains both truth directions

Definition to say aloud. Logical equivalence $A \leftrightarrow B$ requires its two conditions to have the same truth value: both true or both false.

Your expression is:

```
(wr == 1) <-> (oe == 0);
```

When `wr` is one, `oe` must be zero so both conditions are true. When `wr` is zero, the left condition is false, so the right condition must also be false: `oe` must be one. For these two-state one-bit fields, the allowed pairs are exactly $(wr, oe) = (1, 0)$ and $(0, 1)$.

<code>wr</code>	<code>oe</code>	<code>wr==1</code>	<code>oe==0</code>	Equivalence
0	0	false	true	false
0	1	false	false	true
1	0	true	true	true
1	1	true	false	false

This is stronger than $(wr == 1) \rightarrow (oe == 0)$, which would also allow $(0, 0)$. In your two-state model, $wr \neq oe$ describes the same permitted bit pairs as the equivalence expression. That shortcut relies on these operand types and the particular conditions; do not generalize it to arbitrary four-state or multibit expressions.

The names suggest write and output-enable controls, but the relation itself only guarantees opposite bit values. It does not determine whether a given bit level means an active read operation in a particular hardware interface.

6.9.2 What the commented if/else actually requires

Definition to say aloud. A conditional constraint activates one set of relations when its condition is true and the alternative set when it is false; all selected relations participate in the same solve.

Your commented block is:

```
constraint write_read {
  if (wr == 1) {
    waddr inside {[11:15]};
    raddr == 0;
  } else {
    raddr inside {[11:15]};
  }
}
```

In the write branch, five write addresses are allowed, and the read address is exactly zero. In the other branch, the read address has five choices but the write address is not restricted by this block. It remains a four-bit random variable and may take any of its sixteen representable values unless another active rule restricts it.

If the intended convention is "the inactive address is always zero," add `waddr == 0;` in the `else` branch. This is an intentional strengthening of your source, not a formatting correction. Your original omission may be entirely reasonable if the interface ignores that address during reads.

Use `==` for the equality requirement. `raddr = 0` is a procedural assignment; it is not the declarative equality needed here. The `if/else` form does not convert the contents into procedural code.

6.9.3 A missing restriction changes more than printed values

With only this block, ordinary `rand` fields, and no additional weighting or ordering, the write branch has $5 \times 1 = 5$ address combinations. The other branch has $16 \times 5 = 80$ combinations. The unordered legal solution space contains 85 triples $(wr, waddr, raddr)$, so the derived probability of $wr == 1$ is $5/85 = 1/17$.

Adding `waddr == 0` in the `else` branch leaves five combinations in each branch, giving ten in total and an even branch split in that isolated model. Another way to express an even choice of operation while retaining an unconstrained inactive address is `solve wr before waddr, raddr;` with ordinary `rand` variables.

These counts explain the mechanism; they are not predictions for your complete weighted final class. That class has other `dist` rules, and the entire address block is currently commented out. A rule inside `/* ... */` contributes no constraints at all.

6.9.4 Complete example: test the missing address restriction

This experiment activates your original address rule and puts the optional extra restriction in a separate block. With that extra block disabled, an unused write address of 7 is legal during a read. Enabling the extra block makes the same request fail. It also checks all four reset/enable pairs against the unweighted implication truth table.

```

class address_item;
  rand bit wr, rst, ce;
  rand bit [3:0] raddr, waddr;
  constraint implication { (rst == 0) -> (ce == 1); }
  constraint write_read {
    if (wr == 1) {
      waddr inside {[11:15]};
      raddr == 0;
    } else {
      raddr inside {[11:15]};
    }
  }
  constraint zero_unused { (wr == 0) -> (waddr == 0); }
endclass

module address_demo;
  initial begin
    address_item g;
    bit ok;
    g = new();
    g.zero_unused.constraint_mode(0);
    if (!g.randomize() with { wr == 1; waddr == 11; })
      $fatal(1, "Legal write failed");
    if (g.raddr != 0) $fatal(1, "Write read-address is not zero");
    if (!g.randomize() with { wr == 0; waddr == 7; })
      $fatal(1, "Original read branch should allow waddr 7");
    if (!(g.raddr inside {[11:15]}))
      $fatal(1, "Read address outside required range");
    g.zero_unused.constraint_mode(1);
    if (g.randomize() with { wr == 0; waddr == 7; })
      $fatal(1, "Strengthened read branch should reject 7");

    for (int r=0; r<2; r++) begin
      for (int c=0; c<2; c++) begin
        ok = g.randomize() with { rst == r; ce == c; };
        if (ok != ((r != 0) || (c == 1)))
          $fatal(1, "Implication truth table is wrong");
      end
    end
    $display("PASS address_demo: branches and implication");
    $finish;
  end
endmodule

```

The two deliberate failures are the read request with `waddr=7` after the added rule is enabled, and the forced reset/enable pair $(0,0)$. The original address branch is preserved in the experiment; the stronger inactive-address convention is explicitly controlled.

Reference: [IEEE 1800-2017](#), §§11.4.7, 18.5.7, 18.5.10.

6.10 Constraint modes and your final program

6.10.1 What constraint_mode changes

Definition to say aloud. `constraint_mode()` controls whether a named constraint block participates in later randomization calls, or reports that block's current enabled state.

```
g.equivalence_const.constraint_mode(0); // disable this block
g.equivalence_const.constraint_mode(1); // enable this block
state = g.equivalence_const.constraint_mode(); // query: 0 or 1
```

The form with an argument changes the mode; the form without an argument queries it. Newly constructed objects begin with their constraints enabled. For your ordinary nonstatic constraint blocks, a change on one object applies to that object. It does not automatically configure every later `new()` object.

Disabling the equivalence does not set `wr` or `oe` to zero, does not freeze them, and does not turn off `value_wr` or `value_oe`. Those fields remain random and their separate distribution blocks remain active. With equivalence off, all four two-bit combinations are permitted by this portion of the model.

Re-enabling a constraint also does not immediately repair the current field values. It changes the requirements for the next solve. This matters if you inspect a transaction between toggling a mode and calling `randomize()` again.

6.10.2 The exact effective configuration in your final block

Your testbench constructs `g` once, disables `equivalence_const` once, then executes ten randomization calls. No later statement enables that constraint again. The no-argument query consequently prints **0 on all ten iterations**, assuming the simulation reaches those prints. The query reports a mode flag; it does not report the truth of `(wr==1) <-> (oe==0)` in the latest sample and does not report randomization success.

The effective configuration is:

Element in the final source	Effect during the loop
<code>control_rst, control_ce</code>	Active weighted reset and enable rules.
<code>control_rst_ce</code>	Active; forbids <code>rst=0, ce=0</code> .
<code>value_wr, value_oe</code>	Active independent per-bit weight declarations.
<code>equivalence_const</code>	Present but disabled by its mode call.
<code>write_read</code>	Absent from compiled behavior because it is commented out.
<code>randc bit [3:0] a</code>	Active, unconstrained cyclic field with sixteen values.
<code>raddr, waddr</code>	Active four-bit random fields; no active address restriction.

Because the same object is retained and `a` has an unchanged full domain, ten successful calls cover ten distinct values of its current sixteen-value cycle. The source does not display `a`, so the ten mode-status lines cannot demonstrate that property. Printing only the mode also hides reset/enable legality and the actual write/output-enable combinations.

6.10.3 Complete final example, with observable checks

This version preserves your final active constraints and the commented address block. It changes class/module names to avoid collisions, uses an explicit failure check, and displays the randomized fields as well as the mode. The added checks verify the active implication, the disabled mode, and the first ten samples of a's cycle.

Simulator support. XSim 2024.1 rejects $\leftrightarrow$ inside constraints during elaboration, even when the testbench would later disable that block. The source below retains the valid language operator you are studying. For this two-state example, a temporary local variant using equality of the two Boolean conditions ran successfully; [the verification record](#) identifies that adaptation.

```
class control_item;
  randc bit [3:0] a;
  rand bit ce, rst, wr, oe;
  rand bit [3:0] raddr, waddr;
  constraint control_rst {
    rst dist {0 := 40, 1 := 60};
  }
  constraint control_ce {
    ce dist {1 := 80, 0 := 20};
  }
  constraint control_rst_ce {
    (rst == 0) -> (ce == 1);
  }
  constraint value_wr {
    wr dist {0 := 50, 1 := 50};
  }
  constraint value_oe {
    oe dist {0 := 50, 1 := 50};
  }
  constraint equivalence_const {
    (wr == 1) <-> (oe == 0);
  }
  /* This block is inactive, as in your final source.
  constraint write_read {
    if (wr == 1) {
      waddr inside {[11:15]};
      raddr == 0;
    } else {
      raddr inside {[11:15]};
    }
  }
  */
endclass

module control_demo;
  initial begin
    control_item g;
    bit [15:0] seen;
    g = new();
    g.equivalence_const.constraint_mode(0);
    repeat (10) begin
      if (!g.randomize()) $fatal(1, "Control solve failed");
      if (g.rst == 0 && g.ce == 0)
        $fatal(1, "Implication was violated");
      if (g.equivalence_const.constraint_mode() != 0)
        $fatal(1, "Equivalence unexpectedly enabled");
      if (seen[g.a]) $fatal(1, "a repeated before cycle ended");
      seen[g.a] = 1;
      $display("rst=%0b ce=%0b wr=%0b oe=%0b a=%0d",
        g.rst, g.ce, g.wr, g.oe, g.a);
    end
  end
endmodule
```

Code continued

```

    $display("raddr=%0d waddr=%0d equivalence_mode=%0d",
            g.raddr, g.waddr,
            g.equivalence_const.constraint_mode());
end
$display("PASS control_demo: mode=0 on all ten calls");
$finish;
end
endmodule

```

There is no delay in this final program, matching your last fragment. All calls and prints occur at the same simulation time. The loop index counts generated samples, not clock cycles.

When equivalence is disabled, equal bits such as (0,0) become legal. A ten-sample run is still not guaranteed to show every legal pair. A deterministic test can force each candidate pair with an inline constraint and check whether the solver accepts it.

6.10.4 constraint_mode and rand_mode are different controls

Definition to say aloud. `rand_mode(0)` freezes a random variable for subsequent solves, while `constraint_mode(0)` removes a named relation from those solves.

If `g.a.rand_mode(0)` is called, `a` becomes state for solving purposes and retains its current value. Constraints referring to it remain active unless separately disabled. If `a` is frozen to zero but an active constraint requires `a > 3`, randomization can fail; the solver is not allowed to change the frozen field to satisfy the rule.

Conversely, disabling a range constraint leaves an active `rand` variable eligible for new values. Its type still limits the representable domain, and every other active constraint still applies. This is why disabling only `equivalence_const` leaves `reset/enable` behavior intact.

Inline constraints in `g.randomize()` with `{ ... }` are additional restrictions for that call. They do not override an active hard class constraint. A contradictory inline request is useful in a negative test precisely because the expected result is failure.

6.10.5 Complete mode experiment with deterministic outcomes

This isolated experiment removes the weights so that it tests acceptance of bit pairs directly. The equivalence relation is identical to yours. It also verifies that frozen variables continue to be checked against active constraints. The local loop variables are not class members, so the inline expressions resolve `w` and `o` in the calling scope.

```

class mode_item;
    rand bit wr, oe;
    constraint relation { (wr == 1) <-> (oe == 0); }
endclass

module mode_demo;
    initial begin
        mode_item g;
        bit ok;
        g = new();
        if (g.relation.constraint_mode() != 1)
            $fatal(1, "New constraint should start enabled");
        for (int w=0; w<2; w++) begin
            for (int o=0; o<2; o++) begin
                ok = g.randomize() with { wr == w; oe == o; };
                if (ok != (w != o))

```

Code continued

```

        $fatal(1, "Enabled equivalence truth table is wrong");
    end
end

g.relation.constraint_mode(0);
for (int w=0; w<2; w++) begin
    for (int o=0; o<2; o++) begin
        if (!g.randomize() with { wr == w; oe == o; })
            $fatal(1, "Disabled relation rejected a bit pair");
    end
end

g.wr = 0; g.oe = 0;
g.wr.rand_mode(0); g.oe.rand_mode(0);
g.relation.constraint_mode(1);
if (g.randomize())
    $fatal(1, "Frozen illegal pair should fail");
g.oe = 1;
if (!g.randomize()) $fatal(1, "Frozen legal pair failed");
if (g.wr != 0 || g.oe != 1)
    $fatal(1, "Frozen values changed");
$display("PASS mode_demo: pair legality and frozen fields");
$finish;
end
endmodule

```

There are three deliberately unsuccessful calls in this test: the two equal pairs while equivalence is enabled, and the frozen (0,0) pair. Each failure is checked. The disabled relation accepts all four forced pairs. That gives stronger evidence about the mode's meaning than hoping a short random loop happens to print a forbidden pair.

Local result. The exact source encounters XSim's <-> support limitation. A diagnostic using the two-state equivalent `wr != oe` confirmed the enabled and disabled pair tests, then stopped because XSim accepted the illegal frozen pair. The last check deliberately remains in this language-reference example. A successful result from that tool in this case does not establish that the pair satisfies the active constraint.

Reference: [IEEE 1800-2017](#), §§18.7-18.9.

6.11 A method for debugging randomization

6.11.1 Start with the executed program

Your paste contains six active generator declarations and six active tb modules, plus a commented first program. These are separate experiments. Compiling the entire paste as one source would create duplicate definitions and would not represent any one of the intended tests.

First identify the chosen class and top module. Then mark what is commented out. In the final fragment, the histogram example and the address constraint are comments; the equivalence is compiled but subsequently disabled. Those three situations have different meanings: absent code, an inactive named rule, and an active rule.

Normalize chat transport artifacts such as ` ` and escaped underscores before compiling a copied chat excerpt. Your original attachment already contains ordinary SystemVerilog characters, so it is the preserved source used for this guide. The repeated final block sent later adds no new code beyond that attachment.

6.11.2 Build the legal set before looking at randomness

Use this sequence on a failing call:

1. Check that the handle is constructed and the program compiled without a source error.
2. Write the type domain: for your unsigned four-bit fields, 0 through 15.
3. Substitute current ordinary state such as `min` and `max`.
4. Identify active constraints and active random variables from their mode flags.
5. Intersect ranges and apply relationships, including inline constraints for that call.
6. Check the Boolean return and whether callbacks ran; use data only after success.

For example, `min=16, max=33` is already impossible at step 3. No choice of random seed fixes the empty intersection. By contrast, a valid sample repeated across two new objects is not an unsatisfiable-constraint failure at all; it is a misunderstanding of object lifetime.

If the set is legal but observed frequencies look surprising, inspect weights, branch sizes, and ordering. Do not silently relax hard constraints merely to make the randomizer return success: the generated traffic must still satisfy the intended protocol.

6.11.3 Seed control and reproducibility

Definition to say aloud. A seed initializes pseudorandom generator state so that a controlled test configuration can be reproduced; it is not a portable list of values shared by every simulator.

An object can be seeded using `g.srandom(seed)`; . For replay, retain the simulator and version, source, seed, object construction order, and relevant call history. Changing the number or ordering of construction/randomization operations can change the results. A fixed seed does not make an invalid constraint valid and does not create an exact `dist` quota.

For checks such as range membership, implication truth, cyclic nonrepetition within a defined cycle, or mode state, verify the invariant directly. Use a recorded seed and actual output when investigating a particular random failure. Avoid writing a tutorial assertion that requires one arbitrary sequence of generated numbers across simulators.

6.11.4 A compact correction ledger

Practice detail	Precise interpretation or correction
<code>int</code> status after <code>g=new()</code> in one block	Move the declaration before executable statements.
<code>a > 16</code> with four-bit <code>a</code>	Unsatisfiable; width limits the domain to 0-15.
<code>pre_randomized / post_randomized</code>	Explicit user methods, not automatic callbacks.
Bounds 12 through 33	Effective set is 12-15, with no truncation-based generation.
<code>new()</code> inside a <code>randc</code> loop	Fresh per-object cycle history each iteration.
Two calls assigning the same bounds	Same effective constraint; no range-change demonstration.
<code>randc a, b</code>	Per-field cycles; no full pair-coverage guarantee.
<code>:= 70</code> on <code>[1:3]</code>	Weight 70 for each of the three values.
<code>:/ 90</code> on <code>[1:3]</code>	Total range weight 90, hence weight 30 per value.
<code>(wr==1) <-> (oe==0)</code>	Requires opposite bits while enabled.
Missing <code>waddr</code> rule in the <code>else</code>	Inactive write address remains unconstrained by that block.
Querying disabled equivalence ten times	Prints mode 0 ten times; does not display sample legality.

Reference: [IEEE 1800-2017](#), §§18.6-18.10, 18.13.

6.12 Pronunciation, oral revision, and worked answers

6.12.1 How to pronounce the terms and symbols

These are practical spoken readings for an oral explanation, not special language rules. Capital letters in the pronunciation hints mark the stressed syllable. You can omit underscores and punctuation when explaining the meaning, but say them when someone needs to type the exact identifier.

Written form	Say it aloud
randomization	"ran-duh-mih-ZAY-shuhn"
constraint	"kuhn-STRAYNT"
rand	"rand," rhyming with "hand"
randc	"rand-see," or "random cyclic" when explaining
cyclic	"SY-klik"
randomize()	"randomize," or "call randomize"
inside	"inside"
dist	"dist," as at the start of "distance"

Written form	Say it aloud
implication	"im-plih-KAY-shuhn"
equivalence	"ih-KWIV-uh-luhns"
extern	"eks-TERN," meaning declared externally
pre_randomize()	"pre-randomize"; exact name: "pre underscore randomize"
post_randomize()	"post-randomize"
pre_randomized()	"pre-randomized," with a final d sound
constraint_mode()	"constraint mode"
rand_mode()	"rand mode"
-> in a constraint	"implies"
<->	"if and only if," or "is logically equivalent to"
:= in dist	"colon equals"; weight for each value
:/ in dist	"colon slash"; weight for the whole range
::	"double colon," the scope-resolution operator
==	"equals" as a relation; "double equals" when dictating code
=	"assign," or "single equals" when dictating code
!	"not," or "logical negation"
&& and	"logical and" and "logical or"

Say DUT as "D-U-T" (device under test) and OOP as "O-O-P" (object-oriented programming). The distinction between `pre_randomize` and `pre_randomized` is especially useful to say carefully: that last letter changes an automatic callback name into an ordinary method name in your practice.

6.12.2 Read complete statements, then explain them

`a inside {[3:7]}`; Say: "a is inside the range three through seven, inclusive." Explain: "The legal values are three, four, five, six, and seven." In membership syntax, the numbers are lower and upper bounds. In `bit [3:0]`, say "bits three down to zero" because those are packed bit indices.

`!(a inside {[3:7]})`; Say: "a is not inside the range three through seven." Explain: "Those five values are excluded from the field's representable domain." Do not read it as a request to negate a numerically.

`(rst == 0) -> (ce == 1)`; Say: "Reset equals zero implies chip enable equals one." Then explain: "Whenever reset is zero, chip enable must be one; when reset is one, this rule does not force chip enable." You can say the short names as "R-S-T" and "C-E" when dictating code. "Chip enable" is the conventional expansion of `ce`; the relation itself is the literal bit condition.

`(wr == 1) <-> (oe == 0)`; Say: "W-R equals one if and only if O-E equals zero." Then explain: "The two conditions must have the same truth value, so these one-bit fields must be opposite." For descriptive discussion, `wr` can be read as "write" and `oe` as "output enable," while retaining the interface's actual polarity.

`var1 dist {0 := 30, [1:3] := 70};` Say: "Var one has distribution: zero with weight thirty, and each value one through three with weight seventy." To dictate the syntax, add "colon equals" at each `:=`. Explain that the total is 240 because the range contributes three weights of 70.

`var2 dist {0 :/ 30, [1:3] :/ 90};` Say: "Var two has distribution: zero with weight thirty, and the range one through three with total weight ninety." To dictate the syntax, say "colon slash." Explain that the range gives weight 30 to each of its three values.

`g.equivalence_const.constraint_mode(0);` Say: "Disable the equivalence constraint on g." When dictating it, say "g dot equivalence underscore const dot constraint underscore mode, open parenthesis zero close parenthesis." For the no-argument form, say "query the equivalence constraint's mode." Empty parentheses mean a query in this method form; they are not a request to set zero.

`constraint generator::data` Say: "Define constraint data in class generator," or dictate "constraint generator double colon data." Explain that the definition belongs to the class even though its body is written outside it.

`if (!g.randomize()) $fatal(1, ...);` Say: "If g fails to randomize, terminate with a fatal error." Reading it as "if not g dot randomize" is useful for syntax, but the first version states the intent more clearly.

`%0d, %0b, %0t, and #10` When dictating, say "percent zero d," "percent zero b," "percent zero t," and "hash ten." When explaining, say "print decimal," "print binary," "print time," and "delay ten local time units." The zero requests a minimal field width; it is not the value being displayed.

6.12.3 Say these definitions without looking

Constrained randomization. Selecting values for active random variables that satisfy the active relations, with the applicable distribution and cyclic rules.

State variable. A variable whose current value is treated as fixed input to the solve rather than a value the solver may choose during that call.

Randomization failure. A call that returns zero because the required randomized solution cannot be produced under the current problem; it must be handled before consuming new stimulus.

Cyclic history. The state that tracks progress through a `randc` variable's current permutation; in these examples it belongs to the retained object and field.

External constraint. A named class constraint whose body is defined outside the class using the class-qualified name.

Distribution weight. A relative selection preference over allowed values, subject to the other active constraints, rather than an exact finite-run count.

Constraint mode. Whether a named relation is considered in a later solve; querying it returns the mode flag.

6.12.4 Predict the legal set

Problem 1. A four-bit unsigned `rand a` must satisfy `a > 3` and `a < 7`. What can it become?

Answer. Exactly 4, 5, or 6. Strict inequalities exclude both 3 and 7. There are three legal values, not five.

Problem 2. The same field has `a inside {[12:33]}`. Does the call necessarily fail because 33 is too large?

Answer. No. Values 12 through 15 satisfy both the type and the constraint. It fails if the required interval has no representable value, as with 16 through 33.

Problem 3. `!(a inside {[3:7]})` is active. Are -1, 2, 7, and 8 legal?

Answer. Of those four candidate integers, 2 and 8 are legal. Seven is explicitly excluded. Minus one is not representable as that unsigned four-bit field's numerical value. An assignment of -1 would convert to the bit pattern 15, which is a separate conversion question.

Problem 4. Both `a inside {[0:3]}` and `a inside {[12:15]}` are active. Does the later block win?

Answer. No. Both hard constraints apply, the intersection is empty, and randomization fails.

6.12.5 Trace the object and callback behavior

Problem 5. A retained `randc` object with values 12-15 produces 13, 12, 15, 14, 14. Is the repeated 14 necessarily a bug?

Answer. No. The first four values complete a permutation; the next value can begin a new permutation with 14. The repeated boundary value does not violate the within-cycle rule.

Problem 6. You create a new object before each draw and see 13 twice in a row. Has `randc` failed?

Answer. No. Each ordinary object has its own cycle history. These are first draws from different objects.

Problem 7. Your class defines `post_randomized()`, but the testbench only calls `randomize()`. Will the helper print automatically?

Answer. No. The automatic hook has the exact name `post_randomize()`. Your helper requires an explicit call.

Problem 8. An object succeeds once, then fails because `min` exceeds every representable value. What happens to its fields and callback counters?

Answer. The failed solve leaves the random fields at their preceding values. `pre_randomize()` has run and may have incremented an attempt counter. `post_randomize()` does not run, so a success counter and a derived field updated only there are unchanged. This assumes the pre-hook itself did not assign those data fields.

6.12.6 Work the weights and control relations

Problem 9. Derive the probability of zero for `var1 dist {0 := 30, [1:3] := 70}` in isolation.

Answer. The total weight is 240, not 100. Zero receives $30/240=12.5\%$. Each nonzero value receives about 29.17%.

Problem 10. Derive the probability of zero for `var2 dist {0 :/ 30, [1:3] :/ 90}` in isolation.

Answer. The total weight is 120. The range distributes its weight evenly, so all four values have weight 30 and probability 25%.

Problem 11. `(rst==0) -> (ce==1)` is active and another constraint forces `ce==0`. What must reset be?

Answer. `rst` must be one. Otherwise the antecedent would be true and its required consequent false. This follows from joint constraint solving.

Problem 12. Equivalence is active. Is `(wr, oe)=(0,0)` allowed? What changes when only equivalence is disabled?

Answer. With equivalence on, `(0,0)` is forbidden because its two conditions differ in truth value. With the block off, that pair is permitted by the remaining independent write/output-enable distribution constraints. It is not guaranteed to appear in ten calls.

Problem 13. Uncomment your address block and choose `wr=0`. Must `waddr` be zero?

Answer. No. The `else` branch restricts only `raddr`. Add `waddr == 0` if that inactive-address convention is required.

Problem 14. A mode query prints zero. Does that mean randomization failed or that `wr==oe`?

Answer. Neither follows. It means the named constraint block is disabled. Inspect the randomization return value for success and inspect the fields for their values.

6.12.7 One final trace exercise

Consider this sequence for two `rand` bits with your equivalence constraint:

```
g = new();
g.equivalence_const.constraint_mode(0);
g.wr = 0;
g.oe = 0;
g.wr.rand_mode(0);
g.oe.rand_mode(0);
g.equivalence_const.constraint_mode(1);
ok = g.randomize();
```

Worked answer. Construction starts with enabled constraints. Disabling equivalence permits the manually assigned equal pair to exist without affecting any solve yet. Turning off both random modes makes those bits fixed state. Re-enabling equivalence restores the opposite-bit requirement but does not change the stored values. The next call must check that fixed pair against the active relation and returns zero. Changing `oe` to one before another call makes the frozen pair satisfy the relation.

This exercise connects object state, constraint state, random-variable state, and the return status. Understanding those four separately resolves most of the questions in your pasted section.

6.13 Source map and verification

6.13.1 How the supplied code maps to this guide

The original text is retained in [randomization practice, 26 September 2026](#). The following line ranges refer to that unchanged file. The later message repeating the final program matches its last topic block; it is covered once here.

Original lines	Topic in your practice	Explanation
1-59	Transaction fields, failure question, exclusions, <code>new()</code> in a loop	6.1-6.4
61-93	External <code>rand</code> constraint and display method	6.5
95-136	<code>randc</code> , helper names, bounds 12-33, fresh objects	6.4 and 6.6
139-187	External <code>randc</code> variant; five-value and four-value ranges	6.5
189-248	Distribution comments; <code>SPACE 1</code> and <code>SPACE 2</code> at the same bounds	6.4, 6.6, and 6.7
250-280	Commented first program with <code>:=</code> and <code>:/</code>	6.7
281-334	Weighted controls, implication, equivalence, commented addresses, mode query	6.8-6.10

The closest saved course examples are [Part 22](#), [Part 23](#), [Part 24](#), [Part 25](#), [Part 26](#), [Part 27](#), [Part 28](#), and [Part 29](#). Their saved source snapshots remain separate from the corrected teaching examples here.

6.13.2 Language references

Primary language reference. IEEE Std 1800-2017, *IEEE Standard for SystemVerilog: Unified Hardware Design, Specification, and Verification Language*, DOI [10.1109/IEEESTD.2018.8299595](https://doi.org/10.1109/IEEESTD.2018.8299595). The existing local reference used for the OOP guide was consulted directly. The explanations and examples here use original wording; they are not quotations from the standard.

The relevant clauses are §9.3.1 for block declarations; §11.4.7 for logical implication/equivalence; §11.4.13 for membership; §18.4 for `rand` and `randc`; §18.5.1 for external constraints; §18.5.4 for distributions; §§18.5.6-18.5.7 for conditional constraints; §18.5.10 for ordering; §18.6 for randomization and callbacks; and §§18.7-18.10 for inline constraints and mode controls. §18.13 covers random stability and seeding.

Current standard access. [IEEE 1800-2023 standard page](#) identifies the later edition and links to the IEEE Get Program. The chapter-specific references above identify the 2017 edition actually consulted; the standards landing page alone was not used as evidence for detailed language semantics.

Supplementary primary discussion. [Accellera SV-EC correspondence on solve-before and cyclic variables](#) gives committee context for cyclic-variable ordering. Committee correspondence can include competing interpretations; the normative language reference takes precedence.

6.13.3 Verification scope

The eight complete examples printed in this guide are saved as individual [SystemVerilog source files](#). The [verification record](#) records the actual simulator, command sequence, results, expected failure diagnostics, and measured histogram. Predictions elsewhere are explicitly marked as derived and do not claim to be simulator transcripts.

The examples check excluded ranges and nonrandom fields; two complete independent cyclic sequences; external constraints; callback counts through success, failure, and recovery; distribution sample accounting; the final active control configuration; the distinction between constraint and variable modes; and conditional address and implication rules. The original pasted programs are retained as learning evidence and are not claimed to compile as a single combined source.

Recorded local result: five of eight exact examples passed in AMD Vivado Simulator 2024.1, software build 5076996. Examples 01, 02, 03, 05, and 08 passed unchanged. Example 04 correctly detected a failed-solve field-preservation discrepancy. Examples 06 and 07 could not elaborate because that version does not support `<->` in constraint expressions. Those three exact examples are not reported as passing.

A separate adapted run of Example 06 passed with Boolean-condition equality in place of `<->`. A diagnostic of Example 07 using `wr != oe` passed the enabled/disabled pair checks, then exposed an additional discrepancy: with both random fields frozen, XSim accepted a pair that violated the active relation. These adaptations are tool investigations, not replacements for the preserved reference examples. They do not establish support for the original operator.

The histogram run recorded `var1` counts **1527, 3481, 3474, 3518** and `var2` counts **2955, 3021, 2931, 3093** for values 0 through 3. It recorded 8408 writes and 8362 reads out of 12,000 calls. Those are measured counts from one run; the separately derived target counts in 6.7 describe expectations.

The truth tables, interval counts, and probability calculations were checked separately by enumeration and arithmetic. Exact random sequences and exact finite-sample percentages are not acceptance criteria. The checks target selected language semantics, with the outcomes and tool limitations stated above; they do not establish cross-simulator conformance or DUT functional verification.